

Sub-Agents — Contexto isolado

Agentes especializados, cada um com suas ferramentas, permissões e system prompt próprios. Definidos como arquivos .md em .claude/agents/.

Especialização

Frontmatter YAML

Tool restrictions

Task tool

O que são Sub-Agents?

Instâncias especializadas do Claude Code que rodam com contexto totalmente isolado — cada uma com seu conjunto de ferramentas, permissões e instruções.

Especialização

Cada sub-agent é expert em um domínio: segurança, testes, docs, deploy. Foco total, sem distração do contexto geral.

Isolamento

O contexto é completamente separado do agente pai. Sem acesso a conversas anteriores nem ao histórico da sessão.

Permissões granulares

Cada agente tem sua própria lista de ferramentas permitidas. Um agente de docs pode não ter Bash, por exemplo.



Insight-chave: o agente principal (orquestrador) invoca sub-agents via Task tool para delegar tarefas complexas com contexto 100% separado.

Orquestrador + Sub-Agents



Agente Orquestrador — contexto completo do projeto · decide o que delegar



security-agent

Análise de vulnerabilidades
Bash(read-only) · Read



test-agent

Gera e roda testes unitários
Bash · Write · Read



docs-agent

Documenta código e APIs
Write · Read



refactor-agent

Refatoração e otimização
Bash · Write · Read

Cada agente = um domínio + apenas as ferramentas que precisa (princípio do menor privilégio)

O arquivo de Sub-Agent

📁 .claude/agents/security-agent.md (projeto) · ~/.claude/agents/ (global)

```
---
name: security-agent
description: >
  Especialista em segurança de código.
  Invoque para auditorias e OWASP checks.
tools:
  - Read
  - Bash(command:"npm audit")
model: claude-opus-4-8
---
```

```
# Agente de Segurança
Você é um especialista em segurança.
Verifique: SQL injection, XSS, secrets
expostos e dependências com CVEs.
Retorne JSON: severity, location, fix.
```

📄 Frontmatter YAML

Metadados do agente: nome, descrição, modelo e ferramentas

🎯 description

Descreve quando o orquestrador deve invocar. Seja específico!

🔒 tools (lista branca)

Só as ferramentas listadas ficam disponíveis. O resto é bloqueado.

🧠 model (opcional)

Modelo diferente por agente. Ex.: Opus para análise profunda.

📄 System prompt (corpo)

O Markdown abaixo do --- é o system prompt. Pode ser rico e detalhado.

Tool Restrictions

Definem exatamente quais ferramentas o sub-agent pode usar — até o nível de argumentos específicos.

SINTAXE NO YAML	O QUE PERMITE	CASO DE USO	RISCO
- Read	Ler qualquer arquivo	Só consulta código	Baixo
- Write	Escrever qualquer arquivo	Docs ou geração de código	Médio
- Bash	Qualquer comando shell	Build, testes, deploy	Alto
- Bash(command:"npm test")	Só este comando exato	Agente que só roda testes	Baixo
- Bash(command:"grep")	Só buscas com grep	Auditoria read-only	Baixo
- WebFetch	Buscar URLs externas	Pesquisa de documentação	Médio

🏆 Regra de ouro: dê ao sub-agent apenas as permissões mínimas necessárias (menor privilégio).

Como invocar um Sub-Agent

Método 1 — Linguagem natural

Você diz o que quer; o orquestrador escolhe o agente pelas descriptions.

"Audite a segurança do módulo de auth" → security-agent

"Gere testes para o UserService" → test-agent

"Documente a API de pagamentos" → docs-agent

@ Método 2 — @mention direto

Use @nome-do-agente para invocar explicitamente um agente.

```
@security-agent analise src/auth/  
@test-agent crie testes para  
src/services/payment.ts
```



@mention garante o agente certo, independente da ambiguidade.

REFERÊNCIA		
Campos do Frontmatter		
<code>name</code>		<code>string</code> · obrigatório
Identificador único, usado para @mention e logs. Use kebab-case sem espaços. Ex.: security-agent.		
<code>description</code>		<code>string</code> · obrigatório
Explica quando o orquestrador deve usar o agente. Seja preciso — o LLM lê isso para decidir. Ex.: "Especialista em OWASP...".		
<code>tools</code>		<code>lista YAML</code> · opcional
Lista branca de ferramentas. Se omitido, herda as permissões padrão do projeto. Ex.: - Read, - Bash(cmd:"test").		
<code>model</code>		<code>string</code> · opcional
Modelo a usar. Omitir herda o do pai. Opus para análises profundas, Haiku para tarefas rápidas e baratas.		
<code>max_tokens</code>		<code>number</code> · opcional
Limite de tokens na resposta. Útil para agentes de resumo ou quando a resposta deve ser concisa. Ex.: 4096.		

Time de Code Review

O orquestrador recebe um PR e paraleliza 3 sub-agents especializados.

```
---
name: reviewer-agent
description: >
  Qualidade geral: nomes,
  complexidade, DRY.
tools:
  - Read
---
# Revisor de Qualidade
Analise nomes, funções
longas e complexidade.
```

```
---
name: security-reviewer
description: >
  OWASP, injeções,
  secrets.
tools:
  - Bash(command:"grep")
  - Read
---
# Revisor de Segurança
Verifique SQL injection,
XSS e secrets expostos.
```

```
---
name: test-reviewer
description: >
  Cobertura e qualidade
  dos testes.
tools:
  - Bash(command:"npm test")
  - Read
---
# Revisor de Testes
Verifique cobertura,
mocks e edge cases.
```



O orquestrador diz "revise este PR" → Task tool invoca os 3 agents em paralelo → consolida os relatórios.

Local vs Global

📁 Local · `.claude/agents/`

```
projeto/
├── .claude/agents/
│   ├── security-agent.md
│   └── test-agent.md
└── src/...
```

- ✓ Commitado no git com o projeto
- ✓ Compartilhado com toda a equipe
- ✓ Ideal para agentes de domínio do projeto

🌐 Global · `~/claude/agents/`

```
~/claude/agents/
├── my-reviewer.md
├── my-docs-writer.md
└── my-committer.md

# disponível em TODOS os projetos
```

- ✓ Disponível em todos os projetos
- ✓ Agentes pessoais do desenvolvedor
- ✓ Ideal para preferências pessoais

⚡ Conflito de nomes: agentes locais têm prioridade sobre os globais.

ALTO IMPACTO

Casos de uso avançados

Pipeline de segurança

Commit → invoca security-agent com o diff → análise via grep patterns → JSON de vulnerabilidades → bloqueia merge se severity=critical.

Documentação automática

Detecta arquivos sem JSDoc → docs-agent (só Write + Read) → gera docs no padrão da empresa → comita.

Gerador de testes

Nova função sem testes → test-agent roda npm test para checar o ambiente → gera casos + fixtures → valida que passam.

Refatoração por métricas

Complexidade ciclomática alta → refactor-agent com escopo no arquivo → refatora mantendo a interface pública → confirma testes.



Automação consistente em todo PR, sem intervenção manual.

Teste rápido

Você quer um agente de auditoria que só possa ler e rodar grep — jamais escrever ou executar outros comandos. Como definir tools?

A) - Bash · - Read · - Write

B) - Read · - Bash(command:"grep")

C) - Bash · - Read

D) omitir tools (herda tudo)

Resposta: B. A lista branca inclui apenas Read e Bash restrito ao comando grep — tudo mais é bloqueado automaticamente. As opções com Bash aberto permitiriam qualquer comando; omitir tools herda todas as permissões do projeto. É o princípio do menor privilégio na prática.

Especialistas com contexto isolado

Sub-Agent = .md

Arquivos em `.claude/agents/` com frontmatter YAML + system prompt.

Task Tool / @mention

O orquestrador delega via Task; @nome invoca diretamente.

Tool Restrictions

Lista branca por agente, até no nível de comando: `Bash(command:"cmd")`.

Local vs Global

Local versionado para a equipe; global para uso pessoal.

PRÓXIMA AULA → AGENT TEAMS: ORQUESTRAÇÃO PARALELA E SÍNTESE DE RESULTADOS